

6. Champagne Tower

Aim

To write a Python program to find the amount of champagne in a particular glass of a champagne tower using **Dynamic Programming**.

Algorithm

1. Start.
2. Pour poured glasses of champagne into the top glass.
3. Store the amount of champagne in each glass using a 2D array.
4. If a glass contains more than 1, keep 1 in that glass.
5. The extra champagne flows equally into the two glasses below it.
6. Repeat this process for each row.
7. Find the amount in the required glass.
8. Stop.

Source Code

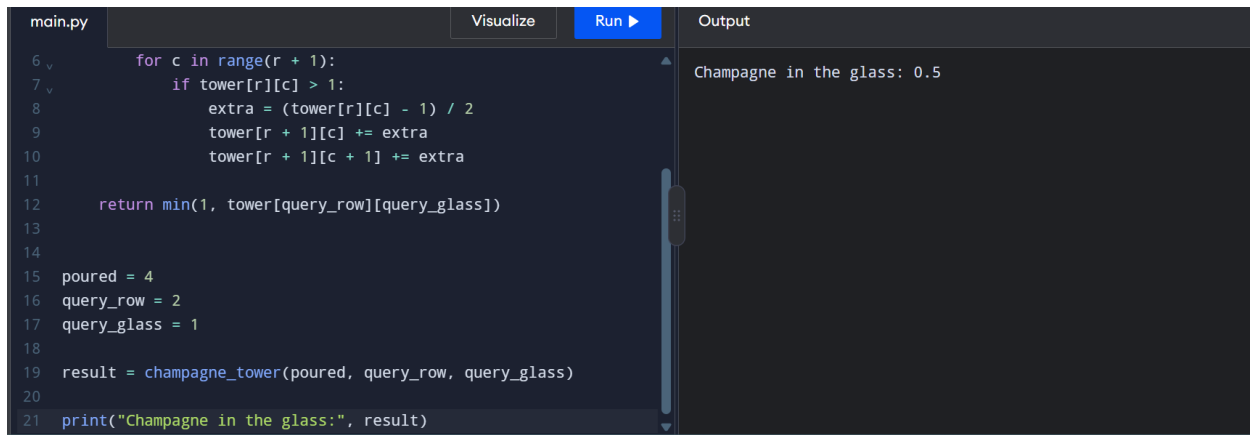
```
def champagne_tower(poured, query_row, query_glass):  
    tower = [[0] * (query_row + 1) for _ in range(query_row + 1)]  
    tower[0][0] = poured  
    for r in range(query_row):  
        for c in range(r + 1):  
            if tower[r][c] > 1:  
                extra = (tower[r][c] - 1) / 2  
                tower[r + 1][c] += extra  
                tower[r + 1][c + 1] += extra  
    return min(1, tower[query_row][query_glass])  
poured = 4  
query_row = 2
```

```
query_glass = 1

result = champagne_tower(poured, query_row, query_glass)

print("Champagne in the glass:", result)
```

Output

A screenshot of a code editor interface. The left pane shows a Python file named 'main.py' with the following code:

```
6   for c in range(r + 1):
7       if tower[r][c] > 1:
8           extra = (tower[r][c] - 1) / 2
9           tower[r + 1][c] += extra
10          tower[r + 1][c + 1] += extra
11
12      return min(1, tower[query_row][query_glass])
13
14
15 poured = 4
16 query_row = 2
17 query_glass = 1
18
19 result = champagne_tower(poured, query_row, query_glass)
20
21 print("Champagne in the glass:", result)
```

The right pane shows the output: "Champagne in the glass: 0.5". The interface includes a 'Visualize' button and a 'Run' button with a play icon.

Complexity Analysis

Let r be the required row.

- **Time Complexity:** $O(r^2)$
- **Space Complexity:** $O(r^2)$

Result

Thus, the **Champagne Tower** problem was successfully implemented using **Dynamic Programming**. The time complexity is $O(r^2)$ and the space complexity is $O(r^2)$.